

به نام خدا
مبانی بینایی کامپیوتر (دکتر سیفی پور)
طراح : نیلوفر مرتضوی، برنا فروهری

پروژه نهایی

نیمسال دوم ۱۴۰۴-۱۴۰۵

مقدمه

با گذشت زمان، بسیاری از عکس‌های قدیمی دچار آسیب‌هایی مانند نویز، خط و خش، لکه، پارگی، کاهش وضوح، تغییر رنگ و از بین رفتن بخشی از جزئیات تصویر می‌شوند. بازسازی این تصاویر یکی از کاربردهای مهم بینایی کامپیوتر و یادگیری عمیق است. هدف در این مسئله آن است که یک تصویر آسیب‌دیده به عنوان ورودی به مدل داده شود و مدل بتواند نسخه‌ای سالم‌تر، واضح‌تر و نزدیک‌تر به تصویر اصلی تولید کند.

در این پروژه، هدف طراحی و پیاده‌سازی یک مدل مبتنی بر معماری انتخابی خودتان برای بازسازی تصاویر آسیب‌دیده است. دانشجویان باید مدل را آموزش دهند، خروجی‌های آن را ارزیابی کنند و نتایج را به صورت کمی و کیفی تحلیل نمایند.



معماری های مهم

در این بخش با معماری های کاربردی که در بازسازی و تولید تصاویر وجود دارد آشنا می شویم.

Autoencoder (AE)

مفاهیم پایه این معماری برای اولین بار در دهه ۸۰ میلادی مطرح شد. هدف آن در ابتدا کاهش ابعاد ورودی به شکل غیرخطی بود. در زمان حال به یک مدل ریاضی پیچیده است که بر روی داده های بدون برچسب و بدون طبقه بندی آموزش می بیند و برای نگاشت داده های ورودی به یک نمایش فشرده شده از ویژگی ها استفاده می شود، و سپس داده ی ورودی را از روی آن نمایش فشرده بازسازی می کند، Autoencoder میگویند. این معماری دارای سه بخش اصلی است.

I. رمزگذار (Encoder)

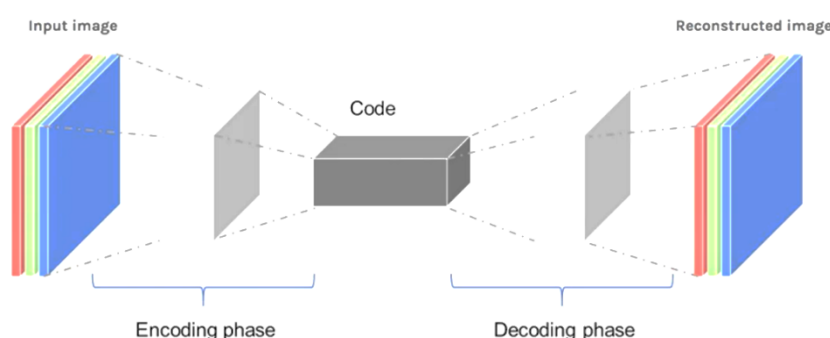
ماژولی که داده های ورودی را به یک نمایش رمزگذاری شده فشرده تبدیل می کند؛ این نمایش معمولاً چندین مرتبه کوچک تر از داده های ورودی است.

II. Bottleneck

این بخش به عنوان نگهبان دانش عمل می کند و جریان اطلاعات از رمزگذار به رمزگشا را کنترل می نماید. طراحی Bottleneck به گونه ای است که بیشترین اطلاعات ممکن از تصویر را ثبت می کند، به طوری که به جای حفظ کامل تصویر، یک نمایش فشرده و معنی دار از آن ساخته می شود.

III. رمزگشا (Decoder)

این فرآیند در واقع دوباره به تصویر جان می بخشد و در سمت دیگر شبکه، مانند یک "بازکننده ی فشرده سازی" عمل می کند. این بخش از یک دنباله ی عملیات Up-Sampling و بلوک های کانولوشنی برای بازسازی خروجی استفاده می کند. شکل ۲، نشان دهنده معماری کلی AE است.



شکل ۲. نمایش کلی Autoencoder

انواع مختلفی از این معماری وجود دارد که در کاربردهای خاصی نظیر کاهش ابعاد، نویززدایی تصویر، تشخیص ناهنجاری و ... استفاده می شوند. یکی دیگر از کاربردهای آن، تولید تصویر است. معماری مورد استفاده برای اینکار، Variational Autoencoders است که با نام اختصاری VAE معروف است. در ادامه به بررسی آن می پردازیم.

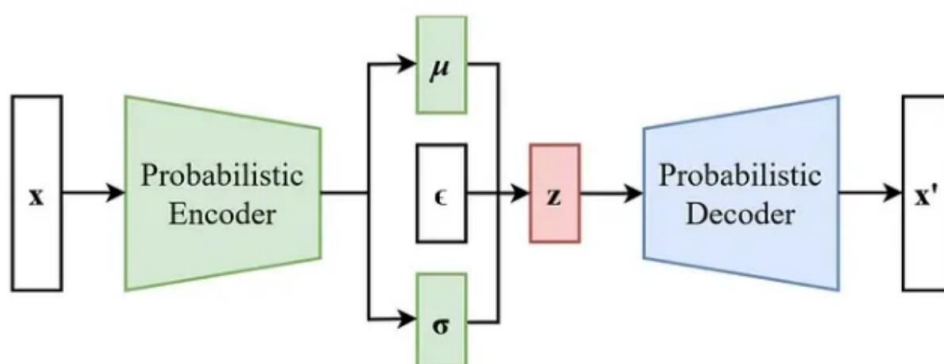
Variational Autoencoders (VAE)

یک نوع معماری شبکه عصبی مولد (Generative) است. در ساختار کلی، VAE همان اجزای اصلی یک AE معمولی را دارد. تفاوت آن در Bottleneck آن است. Bottleneck در AE یک بردار عددی ثابت است که نمایش فشرده‌شده‌ی داده ورودی را نشان می‌دهد. اما در VAE، نه یک بردار عددی ساده، بلکه یک توزیع آماری (معمولاً نرمال گاوسی) است. گلوگاه در VAE نه فقط یک فشرده‌ساز، بلکه یک تنظیم‌کننده‌ی آماری است که شبکه را مجبور می‌کند ویژگی‌های کلی و مهم داده را به صورت توزیعی یاد بگیرد تا هم بازسازی خوبی داشته باشد و هم بتواند داده جدید بسازد. از این رو به Bottleneck آن، Latent Space می‌گویند. پس خروجی Encoder یک توزیع آماری (معمولاً توزیع نرمال یا گاوسی) در فضای نهفته است و Decoder از این توزیع آماری یک نقطه تصادفی نمونه‌گیری می‌کند و تلاش می‌کند که ورودی اولیه را به‌دقت بازسازی کند.

پس مدل باید پارامترهای توزیع را یاد بگیرد. از طرفی میدانیم که فرایند یادگیری پارامترها دارای backpropagation است. علت نمونه‌گیری تصادفی decoder نیز به همین علت است زیرا اگر به طور مستقیم نمونه برداری کند، فرایند مشتق‌گیری در آن غیرقابل انجام است و به همین خاطر با یک نویز نرمال نمونه برداری انجام می‌شود. اینکار باعث میشود تابع z ، نسبت به پارامترهای توزیع، دترمینیستیک باشد و فرایند تصادفی بر عهده ϵ باشد.

$$z = \epsilon \cdot \sigma + \mu$$

شکل ۳، معماری کلی برای توزیع استاندارد را نشان می‌دهد.



شکل ۳. معماری VAE برای توزیع گاوسین

یک معماری تولید کننده دیگر نیز وجود دارد که رویکرد متفاوتی با دو معماری قبلی دارد.

Generative Adversarial Network (GAN)

معماری GAN از دو شبکه‌ی عصبی تشکیل شده است Generator و Discriminator، که به صورت هم‌زمان و از طریق یک فرایند رقابتی (Adversarial Learning) آموزش می‌بینند.

I. مولد (Generator)

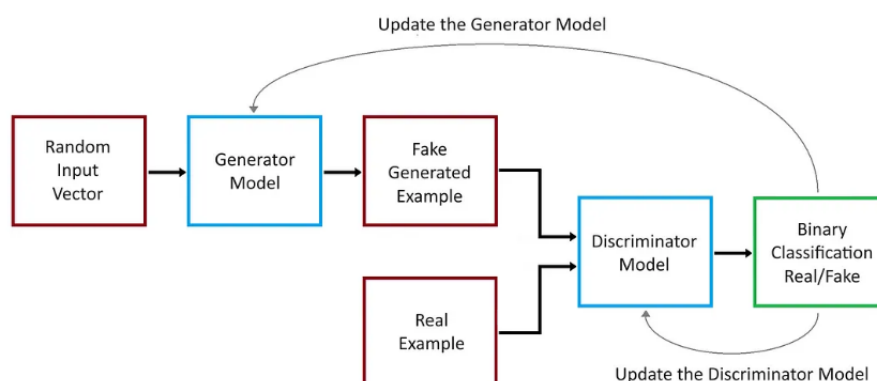
این شبکه نوین تصادفی را به عنوان ورودی دریافت کرده و داده‌ای مانند تصویر تولید می‌کند. هدف آن تولید داده‌ای است که تا حد امکان شبیه به داده‌های واقعی باشد.

II. متمایزکننده (Discriminator)

این شبکه داده‌های واقعی و داده‌های تولیدشده توسط مولد را به عنوان ورودی دریافت می‌کند و سعی دارد بین آن‌ها تمایز قائل شود. خروجی آن احتمال واقعی بودن داده‌ی ورودی است.

این دو شبکه در یک بازی رقابتی قرار دارند. مولد در تلاش است که داده‌هایی تولید کند که تمییزدهنده نتواند آن‌ها را از داده‌های واقعی تشخیص دهد، در حالی که تمییزدهنده سعی دارد در تمایزگذاری بین داده‌های واقعی و ساختگی بهتر عمل کند. پس اگر مولد بتواند داده‌های ساختگی قابل قبولی که نزدیک به داده‌های اصلی است تولید کند، می‌توانیم بگوییم داده تولید شده خوب است.

برای آموزش این معماری، generator از یک بردار نوین تصادفی شروع به ساخت تصاویر می‌کند اما از توزیع داده‌ها اطلاعاتی ندارد. سپس در هر مرحله پس از ساخت، تصویر را به discriminator می‌دهد که تشخیص دهد با چه احتمالی تصویر واقعی است. از نتیجه این مولد به مرور توزیع دیتا را یاد می‌گیرد و این رقابت که داده تولیدی بتواند discriminator را گول بزند تبدیل به یک بازی minmax می‌شود.



شکل ۴. معماری GAN

از کاربرد های آن میتوان به تولید تصاویر رزولوشن بالا، انتقال تصویر از یک حوزه به حوزه دیگر ، augmentation و همچنین پیش پردازش برای مدل های طبقه بند و ... است.

ارزیابی

ارزیابی مدل یکی از مهم ترین کارهایی است که عملکرد مدل را بررسی می کند. در تولید داده، دو نوع ارزیابی وجود دارد.

الف - کیفی:

این ارزیابی توسط متخصصان حوزه ای که مدل در آن آموزش دیده است، انجام می شود. مثلاً اگر قصد تولید تصاویر پزشکی رزولوشن بالا را داشتیم، از چند متخصص برای ارزیابی خروجی دعوت می کنیم و نظرات آن ها را برای خروجی میپرسیم تا از اطلاعاتی از عملکرد مدل به دست بیاوریم.

ب - کمی:

در حوزه بینایی متریک هایی وجود دارند که ما میتوانیم از آنها برای ارزیابی مدل استفاده کنیم. در ادامه به صورت خیلی کوتاه چند تا از آن ها را معرفی می کنیم.

- mse : میانگین مربع اختلاف بین پیکسل های تصویر اصلی و تصویر بازسازی شده
- mae : میانگین قدرمطلق خطاها بین پیکسل ها.
- PSNR : نشان می دهد که چقدر تصویر بازسازی شده شبیه به تصویر اصلی است (برحسب دسی بل).
- SSIM : کیفیت ساختاری تصویر را اندازه گیری می کند (روشنایی، کنتراست، ساختار).

در کاربردهای واقعی نیز از هر دو روش ارزیابی برای ارزیابی مدل ها استفاده می شود.

دیتاست

برای انجام پروژه از دیتاست زیر استفاده خواهد شد:

<https://huggingface.co/datasets/joshuachin/openphoto-restore-dataset>

این دیتاست شامل جفت تصویرهای آسیب دیده و سالم است. به این معنا که برای هر تصویر خراب شده، یک تصویر سالم متناظر نیز وجود دارد. بنابراین پروژه به صورت supervised learning انجام می شود.

برای دریافت دیتاست پیشنهاد می شود از کتابخانه Hugging Face Datasets استفاده کنید. ابتدا بسته مورد نیاز را نصب کرده و سپس دیتاست را بارگذاری نمایید:

```
pip install hf_xet

from datasets import load_dataset

data = load_dataset("joshuachin/openphoto-restore-dataset")

train_dataset = data['train']

test_dataset = data['test']
```

پس از بارگذاری، می‌توانید تصاویر آسیب‌دیده و تصاویر سالم متناظر را از مجموعه‌های `train` و `test` استخراج کرده و برای آموزش و ارزیابی مدل استفاده کنید.

هر نمونه شامل موارد زیر است:

- تصویر ورودی: تصویر آسیب‌دیده
- `Ground truth`: تصویر سالم
- خروجی مدل: تصویر بازسازی‌شده توسط U-Net

می‌توانید برای کاهش زمان آموزش، تصاویر را به اندازه کوچک‌تر مانند 256×256 یا 512×512 تغییر اندازه دهید.

مراحل انجام پروژه

مرحله اول: آماده‌سازی داده‌ها

در این مرحله باید دیتاست دانلود و آماده شود.

۱. تصاویر آسیب‌دیده و تصاویر سالم متناظر را بخوانند.
۲. اندازه تصاویر را یکسان‌سازی کنند.
۳. مقادیر پیکسل‌ها را نرمال‌سازی کنند.
۴. داده‌ها را به سه بخش `train`، `validation` و `test` تقسیم کنند.
۵. `DataLoader` مناسب برای آموزش مدل ایجاد کنند.

مرحله دوم: انتخاب و پیاده‌سازی مدل

در این مرحله باید یکی از معماری‌های مناسب برای بازسازی تصویر انتخاب و پیاده‌سازی شود. مدل انتخابی باید بتواند تصویر آسیب‌دیده را به عنوان ورودی دریافت کرده و تصویر بازسازی‌شده‌ی هم‌اندازه با تصویر ورودی تولید کند.

در گزارش توضیح دهید که چرا این معماری را انتخاب کردید، ساختار کلی آن چیست و چه مزایا و محدودیت‌هایی برای این مسئله دارد.

مرحله سوم: آموزش مدل

پس از پیاده‌سازی مدل، باید فرآیند آموزش انجام شود. موارد زیر را مشخص کنید:

۱. تابع هزینه استفاده‌شده، مانند MSE Loss یا L1 Loss

۲. optimizer، مانند Adam

۳. learning rate

۴. batch size

۵. تعداد epoch ها

۶. اندازه تصویر ورودی

۷. سخت‌افزار مورد استفاده برای آموزش

در طول آموزش باید مقدار train loss و validation loss ذخیره و نمودار آن‌ها رسم شود.

مرحله چهارم: ارزیابی مدل

مدل آموزش‌دیده باید روی test set ارزیابی شود. برای ارزیابی کمی، حداقل دو معیار زیر باید محاسبه شوند:

۱. PSNR

۲. SSIM

همچنین می‌توان از معیارهای زیر نیز استفاده کرد:

• MSE

• MAE

نتایج باید به صورت جدول ارائه شوند.

مرحله پنجم: نمایش خروجی‌ها

حداقل ۵ نمونه از خروجی مدل را نمایش دهید. برای هر نمونه باید سه تصویر کنار هم نمایش داده شود:

۱. تصویر آسیب‌دیده ورودی

۲. تصویر سالم اصلی یا Ground Truth

۳. تصویر بازسازی‌شده توسط مدل

خروجی های مورد انتظار

در پایان پروژه، موارد زیر باید تحویل داده شوند:

۱. کد پروژه

کد باید شامل بخش های زیر باشد:

- دانلود یا خواندن دیتاست
- preprocessing
- تعریف مدل
- آموزش مدل
- ارزیابی مدل
- تولید خروجی روی تصاویر تست
- رسم نمودار $loss$
- محاسبه PSNR و SSIM
- حداقل عملکرد قابل قبول:

$PSNR \geq 20 \text{ dB}$

$SSIM \geq 0.65$

کد باید قابل اجرا باشد و مسیر فایل ها به صورت واضح مشخص شده باشد.

۲. گزارش پروژه

گزارش باید شامل بخش های زیر باشد:

۱. توضیح preprocessing
 ۲. توضیح معماری پیاده سازی شده
 ۳. توضیح تابع هزینه و تنظیمات آموزش
 ۴. نمودار $train loss$ و $validation loss$
 ۵. نتایج کمی مدل
 ۶. نمونه خروجی های تصویری
 ۷. تحلیل نتایج
 ۸. مشکلات، محدودیت ها و پیشنهاد برای بهبود
- در گزارش حتماً نام اعضای گروه و شماره دانشجویی نوشته شود.

نمره امتیازی:

۱. اگر از یک UI استفاده کنید، تا ۱۰ درصد نمره امتیازی دریافت می کنید. (دقت شود نمره این بخش برای مدلی است که خروجی های مورد انتظار را داشته باشد).

نکات نهایی

۱. ددلاین پروژه تاریخ ۲۰ تیر است.
۲. این پروژه دارای ارائه می باشد و باید آمادگی ارائه آن را داشته باشید.
۳. کسب حداقل نصف نمره پروژه برای گذراندن درس الزامی است.
۴. پروژه را می توان به صورت انفرادی یا در گروه های دو نفره انجام داد. حتما در گزارش کار نام اعضا و شماره دانشجویی نوشته شود؛ در غیر این صورت نمره برای اسامی نوشته نشده تعلق نمی گیرد.
۵. در صورت ننوشتن گزارش کار، نمره ای به پروژه تعلق نمی گیرد.
۶. فایل کد و گزارش کار را در پوشه ای با نام زیر در سامانه آپلود کنید.

CV-FinalProject-std#1-std#2

موفق باشید